


**Nombres Completos:**

Josseline Jhomayra Granda Chacha

**Materia:**

Herramientas Informáticas para el Despliegue de Diagramas

**Ciclo/Carrera:**

3ro Ciclo de Desarrollo de Software “A”

**Tema:**

Análisis de las peticiones HTTP

**Fecha de entrega:**

05/10/2026

**Docente:**

ING. Jhostin Vacacela

**Ciudad:**

Cuenca

## 1. Objetivo

Crear una API sencilla en NestJS para administrar usuarios usando un array como almacenamiento temporal. El proceso incluyó definir la estructura de los usuarios, conectar el array con el servicio, revisar las rutas del controlador y preparar peticiones HTTP para probar las operaciones desde Postman.

## 2. Estructura utilizada

Se creó una clase llamada Temporal para definir los datos de cada usuario. Cada registro contiene un identificador, un nombre y una edad. Después, se declaró un array llamado usuarios con algunos datos iniciales.

| Elemento | Función |
|---|---|
| **Temporal** | Define id, nombre y edad. |
| **usuarios** | Guarda los registros en memoria. |
| **DTO** | Define los datos que llegan al crear o actualizar. |
| **Service** | Contiene la lógica para administrar el array. |
| **Controller** | Recibe las peticiones HTTP. |

## 3. Proceso realizado

**Definir los datos.** Se organizó la clase Temporal con los campos id, nombre y edad.

**Crear el array.** Se preparó el array usuarios con tres registros iniciales: Juan, María y Pedro.

**Completar los DTO.** Se declararon nombre y edad en CreateUsuarioDto y se revisaron los campos para actualización.

**Conectar el servicio.** Se revisaron los métodos create, findAll, findOne, update y remove para operar sobre los usuarios.

**Iniciar la aplicación.** La terminal mostró que NestJS inició correctamente y registró las rutas del controlador.

**Probar las peticiones.** Se prepararon solicitudes en Postman para consultar, crear, modificar y eliminar registros.

## 4. Peticiones HTTP utilizadas

Las peticiones permiten comprobar las funciones de la API. Para las rutas mostradas en el proyecto, el prefijo utilizado es /usuario. El puerto habitual en la configuración mostrada es 3000.

| Método | Ruta de ejemplo | Uso |
|---|---|---|
| GET | http://localhost:3000/usuario | Consultar todos los usuarios. |
| GET | http://localhost:3000/usuario/2 | Consultar el usuario con id 2. |
| POST | http://localhost:3000/usuario | Crear un usuario enviando nombre y edad. |
| PATCH | http://localhost:3000/usuario/2 | Actualizar parcialmente el usuario con id 2. |
| DELETE | http://localhost:3000/usuario/2 | Eliminar el usuario con id 2. |
| HEAD | http://localhost:3000/usuario | Solicitar cabeceras sin el cuerpo de la respuesta; depende de la configuración. |
| OPTIONS | http://localhost:3000/usuario | Consultar opciones de comunicación; depende de la configuración. |

## 5. Ejemplos para Postman

### Crear un usuario (POST)

Seleccionar POST, escribir http://localhost:3000/usuario y en Body elegir raw y JSON. Ejemplo:

```
{
  "nombre": "Ana",
  "edad": 21
}
```

### Modificar un usuario (PATCH)

Seleccionar PATCH, usar la ruta del usuario y enviar solo el campo que se desea cambiar. Ejemplo:

```
{
  "edad": 22
}
```

### Eliminar un usuario (DELETE)

Seleccionar DELETE y usar la ruta que contiene el id del usuario. Normalmente no hace falta enviar un Body.

## 6. Conceptos importantes

**Array temporal:** Una lista que guarda datos mientras el servidor está funcionando.

**CRUD:** Crear, consultar, actualizar y eliminar registros.

**DTO:** Define qué datos se reciben en una petición.

**Postman:** Permite enviar peticiones HTTP y revisar las respuestas.

**HEAD:** Solicita las cabeceras de la respuesta sin el contenido.

**OPTIONS:** Permite consultar opciones admitidas por una ruta o servidor.

## 7. Resultado y observaciones

La terminal mostró “Found 0 errors” y “Nest application successfully started”, además del registro de las rutas GET, POST, PATCH y DELETE. Esto confirma que la aplicación compiló y arrancó correctamente. Sin embargo, para asegurar que cada operación modifica los datos como se espera, es necesario comprobar las respuestas en Postman.

El array no es una base de datos permanente. Los datos se guardan en la memoria del servidor y se pierden al reiniciarlo, por lo que al volver a iniciar la aplicación se recuperan los registros definidos inicialmente en el código.

## 8. Conclusión

Se organizó una API de usuarios en NestJS usando un array como almacenamiento temporal. Se revisaron la estructura de los datos, los DTO, el servicio y las rutas, y se prepararon peticiones para probar las funciones principales desde Postman. Este método es útil para practicar el funcionamiento de una API antes de conectarla a una base de datos real.